Optimisation des Performances et Rendu React

Objectifs

1. Analyser les goulots d'étranglement d'une application à l'aide de l'onglet Profiler des React Developer Tools.
2. Éviter les rendus en cascade inutiles sur les composants d'interface lourds grâce à `React.memo`.
3. Stabiliser les références des fonctions passées en props aux composants enfants avec le Hook `useCallback`.
4. Mettre en cache les opérations de filtrage ou de calcul coûteuses avec le Hook `useMemo`.

Contexte du Projet : OmniMonitor - Passage à l'Échelle

Le portail **OmniMonitor** est maintenant connecté à une base de données contenant plus de 2 000 stations d'observation. De plus, une horloge globale et un compteur de pings réseau s'actualisent toutes les secondes dans le tableau de bord.

Les opérateurs se plaignent de ralentissements majeurs : chaque fois que l'horloge globale avance d'une seconde ou que l'utilisateur tape une lettre dans la barre de recherche, l'interface se fige brièvement. Votre mission est de diagnostiquer le problème et de le corriger sans altérer les fonctionnalités.

Étape 1 : Diagnostic avec le React Profiler

Avant de modifier le code, il faut prouver qu'il y a un problème de performances.

1. Ouvrez votre application dans le navigateur et ouvrez les outils de développement (touche F12).
2. Allez dans l'onglet **Components** (React DevTools) et cliquez sur l'icône d'engrenage (Paramètres) :
    - Dans l'onglet *General*, cochez la case **"Highlight updates when components render"**.
    - Retournez sur l'application : constatez que toute la liste des 2 000 stations clignote en vert à chaque seconde.
3. Allez dans l'onglet **Profiler**.
4. Cliquez sur le bouton d'enregistrement (rond bleu), tapez quelques lettres dans la barre de recherche, puis arrêtez l'enregistrement.

5. Analysez le Flamegraph : identifiez que le composant parent (`Dashboard`) re-rend systématiquement l'ensemble des enfants (`StationCard`), même ceux qui ne sont pas concernés par la recherche.

Étape 2 : Isolation des Composants avec `React.memo`

Vous allez demander à React de ne plus re-rendre les cartes de stations si leurs props n'ont pas changé.

1. Ouvrez le fichier `src/components/StationCard.jsx`.
2. Importez `memo` depuis `'react'`.
3. Enveloppez l'export de votre composant fonctionnel avec `memo` :

```
JavaScript
export const StationCard = memo(function StationCard({ station, onToggleStatus })
{
  // ... contenu du composant
});
```

4. Retournez dans le navigateur : l'optimisation ne fonctionne pas encore ! Le Profiler indique que le composant se re-rend toujours avec le motif : *"Hook 1 changed, onToggleStatus changed"*.

Étape 3 : Stabilisation des Références avec `useCallback`

Le problème vient du parent `Dashboard` : à chaque fois qu'il se re-rend (à cause de l'horloge ou de la recherche), il recrée une nouvelle fonction `onToggleStatus` en mémoire, ce qui casse l'égalité des props attendue par `React.memo`.

1. Ouvrez le fichier parent `src/pages/Dashboard.jsx` (ou le composant qui gère la liste).
2. Importez le Hook `useCallback` depuis `'react'`.
3. Repérez la fonction qui gère le changement de statut (par exemple `handleToggleStatus`).
4. Enveloppez cette fonction dans `useCallback` :

```
JavaScript
const handleToggleStatus = useCallback((stationId) => {
  dispatch(toggleStationStatus(stationId));
}, [dispatch]); // N'oubliez pas les dépendances !
```

5. Répétez l'opération de profilage : cette fois, les cartes de stations ne doivent plus clignoter lorsque l'horloge tourne. Seules les cartes dont le statut change réellement sont re-rendues.

Étape 4 : Optimisation du Filtrage avec `useMemo`

L'application filtre la liste des 2 000 stations à chaque rendu, même lorsque seul le compteur de pings réseau change. C'est un calcul lourd inutile.

1. Restez dans le composant gérant la liste des stations.
2. Importez le Hook `useMemo` depuis `'react'`.
3. Repérez la variable qui stocke le tableau des stations filtrées :

```javascript
// CODE LENT À OPTIMISER :
const filteredStations = stations.filter(st =>
  st.name.toLowerCase().includes(searchTerm.toLowerCase())
);
```

4. Enveloppez ce calcul dans un `useMemo` pour qu'il ne s'exécute que lorsque `stations` ou `searchTerm` sont modifiés :

```javascript
const filteredStations = useMemo(() => {
  console.log("Calcul du filtrage des 2000 stations...");
  return stations.filter(st =>
    st.name.toLowerCase().includes(searchTerm.toLowerCase())
  );
}, [stations, searchTerm]);
```

5. Ajoutez un bouton incrémentant un simple compteur local dans votre composant. Vérifiez dans la console que le message *"Calcul du filtrage..."* ne s'affiche **plus** lorsque vous cliquez sur ce bouton.

Critères de Validation

Avant de valider ce segment, vérifiez que :

- L'option "Highlight updates" des React DevTools montre que les `<StationCard />` ne clignotent plus lorsque des états externes (horloge, compteur global) changent.
- Les fonctions passées aux enfants optimisés sont stabilisées avec `useCallback`.
- Le filtrage du tableau de bord est mis en cache avec `useMemo` et ne s'exécute que lorsque les termes de recherche changent.

- Le Flamegraph du Profiler montre des barres grises (composants ignorés/mémorisés) pour les cartes non impactées par les mises à jour.